

Bringing the Systems Analysis and Design Course into 21st Century: A Case Study in Implementing Modern Software Engineering Principles

Christopher G. Jones, CPA/PhD
Utah Valley State University
jonescgc@uvsc.edu

Abstract

While today's IT students may have a theoretical knowledge of all the phases in the systems development lifecycle, they often have little exposure to software development core practices. Few are able to build a new system from scratch. This paper examines a pedagogical approach to modern software engineering based on "doing" systems development from day one. Course materials were produced by Agile methodology expert, Alistair Cockburn. Early results from an implementation of the experimental approach to the systems analysis and design course are examined.

1. Introduction

Declining enrollments in computing-related disciplines in US colleges and universities have prompted educators to reexamine the relevance of information technology (IT) curricula [8]. Andriole [2] claims the gap between theory and practice is widening even though concerted efforts have been made by academia to modernize model curricula. Trauth et al. [9] labeled the difference between industry expectations and academic preparation, the "expectation gap". Johnson and Jones [7] refer to the lag between the time industry develops best practices and these practices find their way into the classroom as "academic latency". In a flat world [4] where time and space are no longer serious barriers to entry, the global pace of change exacerbates this latency.

As academic latency increases, traditional approaches to software development education become increasingly irrelevant. The result: *IT majors graduate with a theoretical knowledge of the phases and activities in the system development lifecycle but lack the skills required to build useful applications for today's marketplace.* What is needed is a pedagogical strategy for addressing the relevance gap. This paper presents the preliminary results of a work in progress

to increase relevance and immediacy of software engineering education at a large regional college.

2. Computing at Utah Valley State College

Utah Valley State College (UVSC) is a large regional college located in the U.S. intermountain west. Beginning in 1993, the UVSC School of Business began offering a combined baccalaureate degree in Computer Science/Information Systems (CS/IS). The BS CS/IS shared a common core with three emphases: (a) computer science, (b) information systems, and (b) networking. As the program matured and the number of majors approached 1000, the degree was split. In 2001, the BS CS/IS became a BS in Computer Science and a BS in Information Systems. With the split, the computer science program and faculty were moved to a new School of Computer Sciences and Engineering. The BS Information Systems program and faculty remained in the School of Business.

In 2004, under the leadership of a new academic administration, all computing disciplines were reorganized and clustered in a new School of Technology and Computing (T&C). Today under the auspices of the School of T&C, UVSC offers separate baccalaureate degrees in computer science (CS), information systems (IS), and information technology (IT). The BS in Computer Science is ABET accredited and includes emphases in computer science, software engineering (SE), computer engineering (CE), and networking. The BS Information Systems and BS Information Technology have been designed in accordance with ABET guidelines for these computing subdisciplines [1] with plans to seek initial accreditation in 2008. All computing programs and faculty are housed in the Computer Science building.

As a result of UVSC's strategy of combining all computing education under one roof, there is a close

association between the computer science, information systems, information technology, and multimedia faculty. Many of the computer science faculty have taught information systems courses and vice versa. The author, for instance, has taught both software engineering and systems analysis and design for combined audiences of computer science and information systems majors and to separate audiences of purely CS or IS majors. While model curriculum guidelines [1, 5, 6] provide separate objectives for software engineering and systems analysis, there is a great deal of overlap between these two courses. Primary differences resolve around content depth. Software engineering spends more time on software quality assurance and testing. Systems analysis explores several ways to measure the business value of systems and examines the behavioral aspects of introducing change into an organization.

2. Bridging the Relevance Gap

To bridge the ever growing chasm between practice and theory, two concerns must be addressed: (1) the immediacy of the knowledge transfer of effective practices from industry to academia, and (2) the students' skill level with those effective practices.

2.1. Increasing the Immediacy of Course Content

The primary bottleneck in getting fresh content to students is the text book industry publishing cycle. Depending on publisher, the lead time for getting a textbook on the shelves of the college bookstore can be between one and two years once the draft manuscript has been submitted. Reviews, copyediting, permission, and layout are time intensive. Assuming the manuscript takes six months to a year to write, by the time the book is published, the content can be from one and to three years old. Add to that the time it takes for academia to embrace changes in the field, and the time line can easily stretch to five years or longer. By way of example, the author began teaching object-oriented systems analysis in 1991 with trade books and selected readings. Not until the early 2000s were object-oriented analysis and design texts available for classroom use. And even those were (and continue to be) based primarily on heavy-weight methodologies such as the Rational Unified Process with extensive UML diagramming.

Historically, systems analysis and design was taught at UVSC as a two-course series. Planning, project management, analysis, and requirements elicitation

were taught in the first course. Design, implementation, and maintenance were taught in the second. Beginning Fall 2006, the two systems analysis and design courses were combined into a single course. Coverage of traditional methodologies such as structured analysis/structured design (SASD) was dropped in favor of concentrating on object-oriented analysis and design. This shift in focus to objects-only necessitated a textbook change. After reviewing the available OOAD texts, *Systems Analysis and Design with UML Version 2.0 (2nd Ed.)* by Dennis, Wixom, & Tegarden [3] was selected.

Approximately three weeks before the beginning of the Fall 2006 semester, UVSC colleagues Neil Harrison and Chuck Allison from the Computer Sciences and Networking Department invited the author to explore the possibility of collaborating with Agile methodology expert and original signer of the Agile Manifesto, Alistair Cockburn, on a project to modernize software engineering education for the 21st century. After an initial series of meetings and an examination of the course topics, the author agreed to participate in the project. At that point, the syllabus for INFO 3430 Systems Analysis, Design and Integration was updated to include a second text:

Cockburn, Alistair. (2007). *Software Engineering in the 21st Century: A Guide for Building Software Engineering Competence from Craft, Cooperative Gaming, and Lean Manufacturing*. Just-in-Time draft available in course download area.

Of special interest is the caveat that the text was "just-in-time". Alistair had agreed to stay at least one week ahead of the course schedule in terms of delivering the chapters. He has made good on his promise, delivering drafts to all three of us for review prior to release to the students.

By the third week in the course, students were asked if they wanted to continue with *Software Engineering in the 21st Century* as it was being "streamed" to them or revert to the original hard bound text from Dennis et al. Using the McCarthy "Decider Protocol" for teams introduced in Module 2 of the Cockburn text, the author of this paper proposed using the e-text as the only course textbook. The voting was unanimously in favor of the dropping Dennis, even though the time for selling the text back to the bookstore had passed. In an effort to ameliorate the financial cost of the class decision, arrangements were later made for the bookstore to buy back the unused texts. Unfortunately

many of the students had purchased the text online and were not able to return them for full price.

The e-text has an interesting design philosophy. For each module, a set of slides are prepared for classroom presentation. The slides contain the main points for the topics and act as visual markers for an extended discussion. The text is organized around the slide set and includes a narration for each illustration. Once the module is presented to the class, the slides and accompanying text are uploaded to the course ftp site for student access. Students have commented that they enjoy reading Alistair's slide explanations out of curiosity as to how he might treat the subject differently than was covered in class. Perhaps, this interest in following Alistair's perspective stems from having him present the first module of the course. Students seem to have developed an intimacy with the Alistair that is usually missing with traditional texts that are developed by unknown authors.

2.2. Increasing Student Skill Level with Software Development Core Practices

Employers typically fault higher education for producing students lacking critical skills for building modern information systems [2, 9]. A common complaint often heard is: "Your students know what a system is, but they aren't quite sure where to start when it comes to building one." The premise behind the modernization of the software engineering education at UVSC was that students should be able "to do" systems, not just "know" about them. In order to achieve that objective, the software engineering and systems analysis courses needed to be restructured to focus on skill building. This required that students practice software development and practice it more than once.

With only a one semester time frame to work with, increasing practice levels meant identifying a subset of core software development concepts and practices. Alistair, Neil, and Chuck met several times during the summer 2006 to determine the appropriate set of topics based on IEEE guidelines for software engineering education [6]. A total of 20 modules were identified focusing on the following core practices: (a) working in teams, (b) incremental/iterative development, (c) automated regression testing, (d) use of configuration management, (e) progress reporting through earned-value charts, (f) mid-project retrospectives, and (g) incorporating changing requirements and designs. For a complete description of the experimental course see *Designing an Incremental-Iterative One-Semester,*

Undergraduate Course in Software Engineering by Alistair Cockburn available at http://alistair.cockburn.us/index.php/Designing_an_incremental-iterative_one-semester,_undergraduate_course_in_software_engineering

The 20-module curriculum takes a spiral approach in which students are presented the course material at least three times at increasing greater depth. As early as day one, students meet in teams, receive a system problem and work through analysis, design, and implementation. The INFO 3430 course is taught two days a week, with students meeting 28 times during the semester. Twenty of those class sessions are devoted to the modules. The remaining eight sessions have been scheduled for topics unique to systems analysis and design (e.g. economic feasibility determination using payback analysis, net present value (NPV), and internal rate of return (IRR)) or unique to the systems analysis cohort. Unlike the software engineering cohort using C++, cppunit, and darcs for a development platform, the systems analysis cohort is using C#, NUnit, and Visual Source Safe (VSS). Some class sessions has been devoted to test driven development using NUnit and some to source code control using VSS.

At the time of the writing of this paper, students had completed the first iteration of the development cycle. The first assignment was a simple exercise where students echoed hard coded strings, command line arguments, and keyboard input to the display, complete with unit testing. Students programmed in C#. Since C# has only been in use one year as the pedagogical language in the Information Systems and Technology Department, this presented a challenge. Of the 17 students enrolled in INFO 3430, only two had completed the two-course C# programming series. The other 15 were either familiar with Java or VB 6.0. No instruction was provided in migrating from these earlier teaching languages. Students worked in teams. Those teams having at least one member with a C# or Java background fared the best.

Working with NUnit seemed to be universally problematic. In the second C# programming course, students are introduced to NUnit as a testing framework. With only two students in the INFO 3430 having been exposed to unit testing, the low level of prior knowledge in testing frameworks posed a real problem. Students struggled trying to get NUnit to integrate with Visual Studio. At times they seemed clueless when it came to writing tests. For some reason, source code control was readily understood and easily implemented. Perhaps this was because a

detailed tutorial was provided on Visual Source Safe. Although test framework configuration and unit testing was demonstrated on at least three occasions and a detailed example with source code provided, students still had trouble installing and running NUnit inside Visual Studio. How to write assertions also remained elusive. The next time the course is offered, a detailed tutorial on test framework configuration and unit testing will be added to the course prior to due date for the first software development assignment.

Students have begun to work on the second iteration of the development cycle. In the second iteration, students tackle the entire development process from end-to-end by designing and building a simple up-down counter client-server application. To date students have written the project mission with priority trade-offs, developed use cases for each service to be provided by the system, and identified a release plan. Over the next two weeks students will start and finish construction of the application using the C#/NUnit/VSS development environment. Students will provide project visibility through burn charts.

The third and final iteration of the development cycle is scheduled for the last eight weeks of the course. The first two weeks of the block will be devoted to planning, analysis, and determining what to build. The final six weeks of the block will provide three two-week delivery windows to accommodate three releases of the system. Each release will be accompanied by team reflection with the goal of incrementally improving the software development process. Along the way, students will receive instruction in additional software engineering topics such as software metrics, the SEI Capability Maturity Model (CMM), ISO standards, and software maintenance. It was felt that these topics could be deferred till the end of the course as they would not impact the ability of students to meet software release targets.

3. Acknowledgements

The author is indebted to Alistair Cockburn for his willingness to foster an industry-academia alliance. Thanks also to colleagues Neil Harrison and Chuck Allison for their efforts in bringing this 21st century approach to software development to the UVSC campus.

4. References

- [1] *ABET Criteria for Accrediting Computing Programs*. (October 29, 2005). Retrieved September 15, 2006 from <http://www.abet.org/Linked%20Documents-UPDATE/Criteria%20and%20PP/C001%2006-07%20CAC%20Criteria%209-12-06.pdf>
- [2] Andriole, S. (2006). Business technology education in the early 21st century: The ongoing quest for relevance. *Journal of Information Technology Education*, 5(1), 1-12.
- [3] Dennis, A., Wixom, B. H., & Tegarden, D. (2005). *Systems Analysis and Design with UML Version 2.0 (2nd Ed.)*. Hoboken, NJ: Wiley.
- [4] Friedman, T. (2005). *The World is Flat: A Brief History of the Twenty-first Century*. New York, NY: Farrar, Straus and Giroux.
- [5] Gorgone, J.T., Davis, G.B., Valacich, J.S., Topi, H., Feinstein, D.L., & Longenecker, H.E. Jr. (2002). *IS 2002: Model curriculum and guidelines for undergraduate degree programs in Information Systems*, Association for Information Systems, Atlanta, GA.
- [6] IEEE Computer Society, Association for Computing Machinery, *Software Engineering 2004: Curriculum Guidelines for Undergraduate Degree Programs in Software Engineering*. A Volume of the Computing Curricula Series, August 23, 2004.
- [7] Johnson, D. W. & Jones, C. G. (2006). IS education: The changing complexity of relevance. *Issues in Information Systems*, Vol. VII (1), 188-192.
- [8] Lomerson, W. & Pollacia, L. (2005). Declining CIS enrollment: An examination of pre-college factors. *Proceedings of the 22nd Annual Information Systems Education Conference*, October 2005.
- [9] Trauth, E., Farrell, D., & Lee, D. (1993). The IS expectation gap: Industry expectations vs. academic preparation. *MIS Quarterly*, 7(3), 293-307.

Note:

This paper has covered early results from the software development course restructuring. This experience report along with final lessons learned will be updated for the ICSE conference presentation.